Source-Code & IP Protection DLP — Microsoft Purview

Implementation Solution Guide

Version 1.0.0 · 2026-07-07

Contents

Executive Summary — Source-Code & IP Protection DLP: Microsoft Purview 4
What you get . . . 4
What this delivers . . . 4
The deployment model . . . 5
Outcome . . . 5
1. Solution Overview — Source-Code & IP Protection DLP (Microsoft Purview) 5
What this solution is . . . 5
Why this matters . . . 6
How it works . . . 6
The DLP design (used throughout) . . . 7
What you get . . . 7
2. Requirements Guide — Source-Code & IP Protection DLP (Microsoft Purview) 8
2.1 DLP licensing (E3 baseline; E5 for Endpoint DLP + trainable classifiers) . . . 8
2.2 Least-privilege DLP/compliance role . . . 8
2.3 Security & Compliance PowerShell toolchain . . . 8
2.4 Endpoint DLP device onboarding (for USB/upload coverage) . . . 8
2.5 The Source Code classifier and custom IP detection (security/IP owned) . . . 9
2.6 In-scope channels, a pilot ring, and test-mode discipline . . . 9
2.7 Notification/alert design and exceptions (approved egress) . . . 9
Checklist . . . 9
Global Requirements — Microsoft Purview Solutions 10
1. Microsoft 365 tenant with the required Purview/compliance licensing . . . 10
2. Purview roles (least-privilege) and portal access . . . 10
3. Security & Compliance PowerShell + Graph toolchain . . . 11
4. Data sources & scope in scope . . . 11
5. Compliance, records, privacy & legal stakeholders . . . 11
6. A pilot scope and change control . . . 12
3. Implementation Guide — Source-Code & IP Protection DLP (Microsoft Purview) 12
Step 0 — Connect (all steps) . . . 12
Step 1 — Assess current state (Assess) . . . 13
Step 2 — Design the DLP policy (Design) . . . 13
Step 3 — Create the policy in test mode (Configure) . . . 13
Step 4 — Pilot in test mode (Pilot) . . . 14
Step 5 — Validate the pilot (Pilot → Validate) . . . 14
Step 6 — Enforce and expand (Expand) . . . 14

5. Testing Guide — Source-Code & IP Protection DLP (Microsoft Purview) 15
Test 1 — Prerequisites and enablement are in place . . . 15
Test 2 — Discovery inventory is complete . . . 15
Test 3 — Create dry-run makes no changes . . . 15
Test 4 — Policy exists in test mode across the intended channels . . . 15
Test 5 — The rule detects source code . . . 16
Test 6 — Endpoint egress is evaluated (where licensed) . . . 16
Test 7 — External email egress is evaluated . . . 16
Test 8 — Validation gate passes on the pilot (test mode) . . . 16
Test 9 — False positives are tuned out . . . 16
Test 10 — Enforcement blocks as designed (deliberate, after approval) . . . 16
Test 11 — Reporting evidence is produced . . . 16
6. Troubleshooting — Source-Code & IP Protection DLP (Microsoft Purview) 17
The policy matches nothing / no matches in reports . . . 17
Endpoint (USB/upload) activity isn't detected . . . 17
The policy blocks legitimate developer activity (false positives) . . . 18
Enforcement is blocking developers unexpectedly . . . 18
Policy tips don't appear . . . 18
`DLPCODE-Set-DlpPolicy.ps1 -Commit` fails . . . 18
`DLPCODE-Validate.ps1` reports a channel/mode wrong . . . 18
7. Best Practices — Source-Code & IP Protection DLP (Microsoft Purview) 19
8. Deliverables Summary — Source-Code & IP Protection DLP (Microsoft Purview) 20
Documentation . . . 20
Named artifacts . . . 21
PowerShell scripts . . . 21
What the buyer receives . . . 22
Outcome . . . 22
Appendix A — PowerShell Scripts 23

Executive Summary — Source-Code & IP Protection DLP: Microsoft Purview

Source code and intellectual property are among the organization's most valuable — and most portable — assets. A developer emails a snippet to a personal account, copies a repository to a USB drive, uploads proprietary designs to a personal cloud, or pastes a key algorithm into a chat, and by default nothing detects that the content is source code or IP, and nothing stops it from leaving. You cannot claim the crown jewels are protected when they can walk out through everyday actions. This solution closes that gap with **Microsoft Purview Data Loss Prevention plus classifiers** that detect and protect source code and IP across email, collaboration, and endpoints — configured, piloted in test mode, validated, and defensible.

This is a complete, do-it-today package written for an administrator implementing code/IP DLP for the first time. It delivers the capability end-to-end — from reading existing policies, the Source Code classifier, and Endpoint DLP onboarding, through designing detection and channels and creating the policy **in test mode** on a developer pilot ring, to tuning false positives against real developer activity, validating, and only then moving to enforcement as a deliberate, approved step — with every policy, rule, and pilot ring named once and reused everywhere. Because DLP enforcement blocks real developer workflow, the guide never enforces on day one: it starts in test mode and gates enforcement behind an explicit, separate action. Because capability availability (Endpoint DLP, trainable classifiers) is licensing-dependent and the portal is evolving, the guide cites the method and flags every capability "verify your licensing/tenant."

What you get

- A plain-language Solution Overview of code/IP DLP — the Source Code classifier and custom IP types, the

channels (email, SharePoint/OneDrive, Teams, endpoint USB/upload), policy tips, and test vs enforce mode.

- A Requirements Guide with verify / implement-if-missing for every prerequisite: DLP licensing (E3

baseline; E5 for Endpoint DLP + classifiers), the least-privilege DLP role, the S&C PowerShell toolchain, Endpoint DLP device onboarding, the Source Code classifier and custom IP detection, the in-scope channels and pilot ring, test-mode discipline, and the notification/exception design.

- An Implementation Guide of precise, ordered steps: assess, design detection + channels + actions, create

the policy in test mode on the pilot ring, pilot and tune, validate, **enforce deliberately**, then expand.

- Five PowerShell tools — Discovery, a prerequisite/health helper, a safe Configuration/Remediation tool

(`-WhatIf` by default; creates in test mode; enforce behind its own action), Validation, and Reporting — built on Security & Compliance PowerShell, **read-only or `-WhatIf` by default with no credentials in code**.

- Testing and Troubleshooting guides with explicit pass/fail gates, plus field-proven Best Practices.

What this delivers

Source code and IP governed from Purview: detected by classifier and custom types and restricted from leaving via email, cloud upload, and USB across the in-scope channels — proven in test mode on a developer pilot ring before any enforcement.

The deployment model

Assess → Design → Configure (**test mode**) → Pilot → Validate → Expand (**enforce deliberately**) — with a test-mode gate before enforcement and enforcement enabled only after validation and approval.

Outcome

The code/IP exfiltration blind spot closed — source code and marked IP can't be emailed externally, copied to USB, or uploaded to personal cloud, developers see a policy tip that teaches, and every attempt is logged — with the determination of what is protected IP and which egress is acceptable left to the security, IP, and legal owners who make it.

1. Solution Overview — Source-Code & IP Protection DLP (Microsoft Purview)

What this solution is

Source code and intellectual property are among the organization's most valuable — and most portable — assets. A developer emails a code snippet to a personal account, copies a repository to a USB drive, uploads proprietary designs to a personal cloud, or pastes a key algorithm into a chat. By default nothing detects that the content is source code or IP, and nothing stops it from leaving. This solution closes that gap with **Microsoft Purview Data Loss Prevention (DLP)** plus **classifiers** that detect and protect source code and IP **across channels**:

- **Detect source code and IP** — use Purview's built-in **Source Code trainable classifier**, plus **custom sensitive information types / keyword dictionaries** for project code-names, internal identifiers, and IP markings, so the policy acts on content that *is* source code or IP, not just files with a certain extension.
- **Protect across channels** — apply the DLP policy across **Exchange (email), SharePoint, OneDrive, Teams**, **and** endpoints/devices (Endpoint DLP) **so exfiltration is governed on the paths that matter: email to external recipients, uploads to personal cloud,** copy to USB/removable media**, and unmanaged browser uploads.
- **Educate and restrict** — show a **policy tip**, notify the user, alert admins, and (after piloting) **block or restrict** the egress action for content that matches, so protection teaches and enforces.
- **Prove it in test mode first** — every DLP policy is created in **test/simulation mode**, so you see exactly what it *would* match and block on real activity before any enforcement affects a developer's workflow.

Everything is done with the **Microsoft Purview portal** (purview.microsoft.com) and **Security & Compliance PowerShell** (`Connect-IPPSSession`) — no third-party tooling. Policies are configured,

piloted in test mode on a small ring first, validated against pass/fail gates, then enforced and expanded.

> ***DLP detects and governs egress; it is not a code-repository security tool.*** *This solution detects source code / IP in the M365 data flow and on endpoints and controls how it can leave. It does not secure the Git server, scan repositories for secrets, or replace developer-platform controls — those are complementary. This solution delivers the Purview DLP half across email, collaboration, and endpoints.*

Why this matters

• **Code and IP leave through everyday actions.** Email, personal cloud upload, USB copy, and chat are all

normal — and all exfiltration paths for the crown jewels. DLP inspects the content and governs the action.

• **Content beats extension as a control.** A renamed file or a pasted snippet defeats extension-based

rules. The Source Code classifier and custom IP types match on what the content *is*.

• **Endpoint DLP closes the USB/upload gap.** Email and SharePoint DLP don't see a copy to a USB stick or a

browser upload — Endpoint DLP does, extending protection to the device.

• **It is defensible and auditable.** "Source code and marked IP cannot be emailed externally or copied to

USB, and every attempt is logged" is a statement you can evidence with a policy and its reports, not a hope.

How it works

1. **Assess.** Read the tenant's current state — existing **DLP policies and rules**, which locations they

cover, the availability of the **Source Code classifier** and relevant **sensitive information types**, and whether **Endpoint DLP** device onboarding is in place — all read-only.

1. **Design.** With security/IP owners, decide the **detection** (Source Code classifier + custom IP

types/keywords), the **channels** (email, SharePoint/OneDrive, Teams, endpoint), the **actions** (restrict/block egress, notify, policy tip, alert), and the **exceptions** (approved repos/tools) — aligned to the IP-classification intent your stakeholders set.

1. **Configure.** Create the `Source-Code & IP DLP – Policy` with its rule(s) across the in-scope

channels in Purview, scoped to the pilot, **in test mode** (no enforcement yet).

1. **Pilot.** Run the policy in **test/simulation mode** on a pilot ring — never enforcing on day one — and

review what it *would* match and block on real developer activity. Blocking egress is **high-impact** for developer workflow; prove the matches first.

1. **Validate.** Confirm the policy detects source code / IP across the intended channels, that policy tips

and alerts fire, and that false positives (build tools, legitimate sharing) are tuned out — against the pass/fail gates.

1. **Expand.** Move the policy from test to **enforce** once validated and stakeholders approve, broaden

scope and channels, and hand ongoing tuning and reporting to operations.

The DLP design (used throughout)

Names below are examples; the **determination** of what is protected IP and which egress to block belongs to your security, IP, and legal stakeholders — this solution provides the technical implementation.

Element	Design intent
DLP policy	`Source-Code & IP DLP – Policy` (Exchange + SharePoint + OneDrive + Teams + Endpoint)
Detection rule	`Source-Code Detection Rule` — Source Code classifier + custom IP types/keywords
Actions	Restrict/block egress (external email, USB copy, cloud/browser upload), notify (policy tip), alert admins
Mode	**Test (with/without policy tips) during pilot; Enforce after validation**
Pilot ring	`Source-Code DLP – Pilot Ring – Stage1` — a small set of developer users/devices
Scope/design map	`Source-Code DLP – Map`
Coverage report	`Source-Code DLP – Coverage Report`

> ***Licensing / plan note — verify against your own tenant.*** *DLP for Exchange/SharePoint/OneDrive/Teams is included with* ***Microsoft 365 E3****;* ***Endpoint DLP****,* ***trainable classifiers*** *(including Source Code), and advanced DLP conditions generally require* ***E5 / E5 Compliance****.* ***Endpoint DLP*** *also requires* ***device onboarding*** *(shared with Microsoft Defender for Endpoint). Confirm the exact capability availability, the classifier/SIT names, and portal paths for *your* tenant in Microsoft's documentation before you design — the DLP surface is evolving. Where a capability is unavailable, scope to what your plan supports and record the rest as* ***deferred****.*

What you get

- This **Solution Overview**, a **Requirements Guide** (every prerequisite with verify / implement-if-missing), a precise **Implementation Guide**, a **Testing Guide** with pass/fail gates, a **Troubleshooting Guide**, and field-proven **Best Practices**.
- **Five PowerShell tools** — Discovery, a prerequisite/health helper, a safe Configuration/Remediation tool (`-WhatIf` by default; creates in test mode), Validation, and Reporting — built on Security & Compliance PowerShell, **read-only or `-WhatIf` by default and with no credentials in code**.

> ***Boundary.*** *What constitutes protected IP, which egress is acceptable (approved repos, partner sharing), and any legal obligation are* ***determinations owned by your security, IP, and legal functions****. This solution provides the technical configuration and the evidence — and never enforces before you validate in test mode — not the determination.*

2. Requirements Guide — Source-Code & IP Protection DLP (Microsoft Purview)

Verify **every** requirement before configuring. Each entry: **Description · Why · Verify · Implement-if-missing.** Category-wide prerequisites are in `global-requirements-purview.md` (the six shared requirements: compliance licensing, least-privilege Purview roles, the S&C PowerShell + Graph toolchain, in-scope data sources, compliance/records/privacy/legal stakeholders, and a pilot scope with change control) — those are referenced here, not repeated.

2.1 DLP licensing (E3 baseline; E5 for Endpoint DLP + trainable classifiers)

Description: A tenant licensed for **Data Loss Prevention** — **Microsoft 365 E3** includes DLP for Exchange/SharePoint/OneDrive/Teams; **Endpoint DLP** and **trainable classifiers** (including the **Source Code** classifier) generally require **E5 / E5 Compliance**. (Global Requirement 1.) **Why:** Cross-channel content DLP works on E3, but the endpoint (USB/upload) coverage and the Source Code classifier that make this solution effective are plan-gated. **Verify:** M365 admin center → `Billing → Licenses`; in the Purview portal confirm **DLP** → **Policies**, **Endpoint DLP**, and the **Source Code** classifier are available. `DLPCODE-Check-Prereqs.ps1` reports the DLP signal. **Verify against your own plan. Implement:** Assign E5/E5 Compliance for the endpoint + classifier coverage; where unavailable, scope to email/collaboration DLP with custom IP types and record endpoint/classifier coverage as **deferred**.

2.2 Least-privilege DLP/compliance role

Description: A least-privilege role that can manage DLP — **Compliance Administrator**, **Compliance Data Administrator**, or the **DLP Compliance Management** role group — in the Purview portal. Reserve **Global Administrator** for one-time enablement only. (Global Requirement 2.) **Why:** Creating/editing DLP policies and rules requires the matching role; DLP enforcement is high-impact, so least privilege matters. **Verify:** Purview portal → `Settings → Roles & scopes → Permissions` shows the DLP/compliance role; `Connect-IPPSSession` succeeds and `Get-DlpCompliancePolicy` runs. **Implement:** Assign the least-privilege role under change control; do not use Global Administrator for day-to-day DLP authoring.

2.3 Security & Compliance PowerShell toolchain

Description: A workstation with **ExchangeOnlineManagement** (provides `Connect-IPPSSession` for `Get-/New-/Set-DlpCompliancePolicy` and `...Rule`, and `Get-DlpSensitiveInformationType`), with a modern-auth / MFA admin. (Global Requirement 3.) **Why:** DLP policies/rules and sensitive information types are read/authored through Security & Compliance PowerShell; the provided scripts run there. **Verify:** `Get-Module -ListAvailable ExchangeOnlineManagement`; `Connect-IPPSSession` succeeds and `Get-DlpCompliancePolicy` returns. `DLPCODE-Check-Prereqs.ps1` reports module/connection state. **Implement:** `Install-Module ExchangeOnlineManagement -Scope CurrentUser`; confirm modern-auth sign-in before running steps.

2.4 Endpoint DLP device onboarding (for USB/upload coverage)

Description: For endpoint coverage (copy to USB/removable media, upload to personal cloud/unmanaged browser), the in-scope devices must be **onboarded to Endpoint DLP** (onboarding is shared with Microsoft Defender for Endpoint), and the relevant **egress activities** enabled in Purview **Endpoint DLP settings**. **Why:** Email/SharePoint DLP can't see a USB copy or a browser upload; Endpoint DLP requires onboarded devices to observe and govern those actions. **Verify:** Purview portal → **Settings** → **Endpoint DLP** shows onboarded devices and the egress-activity settings; confirm the pilot devices are onboarded. **Verify the onboarding path against current Microsoft guidance. Implement:** Onboard the pilot devices to Endpoint DLP and enable the egress activities before authoring the endpoint rule; where endpoint coverage is out of plan (2.1), record it **deferred**.

2.5 The Source Code classifier and custom IP detection (security/IP owned)

Description: The **detection design** — the built-in **Source Code trainable classifier** plus **custom sensitive information types / keyword dictionaries** for project code-names, internal IP markings, and proprietary identifiers — translated into rule conditions and owned/signed off by **security/IP/legal**. (Global Requirements 4 & 5.) **Why:** DLP acts on what you define as source code / IP; which content is protected IP is a business/security **determination**, not a technical default. **Verify:** The Source Code classifier is available; the custom IP types/dictionaries are documented and approved; named approvers recorded in `Source-Code DLP – Map`. `DLPCODE-Discover.ps1` reports available info types. **Implement:** Confirm the classifier availability and author the custom IP types before enforcing; where the determination is incomplete, run in **test mode** and defer enforcement.

2.6 In-scope channels, a pilot ring, and test-mode discipline

Description: The **channels in scope** (Exchange, SharePoint, OneDrive, Teams, Endpoint), a **pilot ring** (`Source-Code DLP – Pilot Ring – Stage1`: a small set of developer users/devices), and the discipline to **start in test mode**. (Global Requirements 4 & 6.) **Why:** DLP keys off channels and scope; enforcement blocks real developer workflow. Prove the policy in test mode on a pilot ring before enforcing tenant-wide. **Verify:** The in-scope channels and pilot ring are agreed; `DLPCODE-Validate.ps1` reports the policy's mode; a change window exists. **Implement:** Define the channels and pilot ring; create the policy in **test mode**; move to enforce only after validation and approval.

2.7 Notification/alert design and exceptions (approved egress)

Description: The **policy-tip text**, **user notification**, and **admin alert** design, plus the **exceptions** — approved repositories, developer tools, partner-sharing paths, and build systems that must not be blocked — agreed with stakeholders. **Why:** Policy tips educate and alerts drive response; well-chosen exceptions keep legitimate developer workflow from being blocked. These are essential to a usable code/IP DLP policy. **Verify:** The tip text, notification/alert recipients, and the approved-egress exceptions are documented in `Source-Code DLP – Map`. **Implement:** Define notifications and exceptions before enforcing; tune them during the test-mode pilot.

Checklist

- DLP licensing confirmed (E3 baseline; E5/E5 Compliance for Endpoint DLP + Source Code classifier, else deferred)

• Compliance/DLP-management role assigned least-privilege

• ExchangeOnlineManagement installed; `Connect-IPPSSession` succeeds; `Get-DlpCompliancePolicy` runs

• Endpoint DLP device onboarding + egress-activity settings in place for pilot devices (or deferred)

• Source Code classifier available; custom IP types/dictionaries documented and approved

• In-scope channels + `Source-Code DLP – Pilot Ring – Stage1` defined; plan **starts in test mode**

• Policy-tip text, notifications/alerts, and approved-egress exceptions captured in `Source-Code DLP – Map`

Global Requirements — Microsoft Purview Solutions

These requirements apply to **every** solution in the **Purview** category (implementing and managing Microsoft Purview, sensitivity & retention labels, DLP, data lifecycle & records, insider risk, communication compliance, audit, data security posture management, and data governance). Verify each before starting a solution's implementation steps.

> ***Vendor boundary (applies to every Purview solution).*** *The **Microsoft Purview portal** (purview.microsoft.com), the **Microsoft 365 admin center**, **Security & Compliance PowerShell** (`Connect-IPPSSession`), and **Microsoft Graph** are authored at exact, current click-by-click / cmdlet detail from Microsoft's public documentation. Microsoft Purview features are heavily **licensing-/plan-dependent** (many require **Microsoft 365 E5** or the **E5 Compliance** add-on; some are in preview) and the portal is **actively evolving** — capability availability and portal paths are **cited or flagged "verify your licensing/tenant"**, never asserted or invented. **Regulatory, records-retention, and legal determinations** (retention schedules, records file plans, legal holds, insider-risk case actions, regulatory applicability) are **deferred to the organization's compliance, records, privacy, and legal functions** — the solutions provide the technical configuration method and evidence, not the determination. Automation is **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets.*

1. Microsoft 365 tenant with the required Purview/compliance licensing

• **Description:** A Microsoft 365 tenant with the **compliance licensing** the capability requires

(E3 for baseline; **E5 / E5 Compliance** for advanced DLP, auto-labeling, insider risk, communication compliance, Audit Premium, DSPM, etc.).

• **Why required:** Most Purview capabilities are plan-gated; the feature won't be available without the

right plan.

• **How to verify:** In the M365 admin center confirm the subscription/add-ons; check the feature is

visible in the Purview portal.

• **How to implement if missing:** Assign/purchase the required licensing before starting.

2. Purview roles (least-privilege) and portal access

- **Description:** Access with the least-privilege **Purview / compliance role** the solution names

(e.g. Compliance Administrator, Compliance Data Administrator, Information Protection, Records Management, Insider Risk, eDiscovery, Audit roles), with Global Administrator only where required.

- **Why required:** Purview uses fine-grained role groups; configuration requires the matching role.
- **How to verify:** Purview portal → **Roles & scopes / Permissions** — confirm the role membership.
- **How to implement if missing:** Assign the least-privilege role group under change control.

3. Security & Compliance PowerShell + Graph toolchain

- **Description:** A workstation with **Security & Compliance PowerShell** (`Connect-IPPSSession` via the

ExchangeOnlineManagement module) and **Microsoft Graph PowerShell**, modern-auth / MFA-capable admin.

- **Why required:** Configuration, automation, and the provided scripts run through these.
- **How to verify:** `Connect-IPPSSession` and `Connect-MgGraph` succeed with the admin account.
- **How to implement if missing:** Install the modules and confirm modern-auth sign-in before running steps.

4. Data sources & scope in scope

- **Description:** The **workloads and data sources** in scope (Exchange, SharePoint, OneDrive, Teams,

endpoints/devices, and — for data governance — the broader data estate) and a **data-classification / sensitivity** intent to drive labels, DLP, and retention.

- **Why required:** Purview policies key off the in-scope workloads and the classification scheme.
- **How to verify:** Confirm the in-scope workloads and the classification/sensitivity approach with

stakeholders.

- **How to implement if missing:** Define the scope and classification intent before authoring policies.

5. Compliance, records, privacy & legal stakeholders

- **Description:** Engagement with the **compliance, records-management, privacy, security, and legal**

stakeholders whose policies the configuration must satisfy (retention schedules, records file plans, regulatory obligations, insider-risk and communication-compliance policy, legal hold).

- **Why required:** Purview configuration must align to enterprise policy and legal/records requirements,

not technical defaults; several capabilities have HR/legal sensitivity.

- **How to verify:** Confirm the applicable policies, the records schedule, and named approvers.
- **How to implement if missing:** Identify the owners and capture the requirements before configuring.

6. A pilot scope and change control

• **Description:** A **pilot scope** (users/sites/policies in simulation or a limited ring) to validate

on before tenant-wide enforcement, and a **change-control** process — most Purview policies support a **simulation / test / disabled** mode that MUST be used before enforcement.

• **Why required:** Every solution follows an **Assess → Design → Configure → Pilot (simulate) → Validate

→ Expand (enforce)** model; DLP, labels, retention, and insider-risk changes are high-impact.

• **How to verify:** Confirm a named pilot scope, that simulation/test mode is available, and a change

window/approval path.

• **How to implement if missing:** Define the pilot and agree change control before enforcing anything.

*The deployment model across every Purview solution is the same: **Assess** → **Design** → **Configure** → **Pilot (simulate/test)** → **Validate** → **Expand (enforce)**, with explicit pass/fail gates and a documented rollback. Automation is delivered as **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets. Regulatory/records/legal determinations are deferred to the relevant governance functions.*

3. Implementation Guide — Source-Code & IP Protection DLP (Microsoft Purview)

Follow these steps in order. The model is **Assess** → **Design** → **Configure** → **Pilot (test mode)** → **Validate** → **Expand (enforce)**. Every write is **created in test mode on** `Source-Code DLP – Pilot Ring – Stage1` **first**, previewed with `-WhatIf`, and reversible. Named objects are introduced once and reused everywhere:

Object	Name
DLP policy	`Source-Code & IP DLP – Policy`
Detection rule	`Source-Code Detection Rule`
Pilot ring	`Source-Code DLP – Pilot Ring – Stage1`
Scope/design map	`Source-Code DLP – Map`
Coverage report	`Source-Code DLP – Coverage Report`

> ***Test mode first, always.*** *A DLP policy in **test mode** matches content and shows what it *would* block across channels (email, SharePoint/OneDrive, Teams, endpoint) but **enforces nothing**. Only **enforce** mode restricts real egress. This solution creates the policy in test mode and moves to enforce as a deliberate, approved step.* ***Verify capability availability (Endpoint DLP, Source Code classifier) against your licensing/tenant.***

Step 0 — Connect (all steps)

Open PowerShell 5.1+ or 7+ and connect with a modern-auth/MFA admin:

```
Connect-IPPSSession                # Security & Compliance PowerShell (DLP policies + rules)
```

Keep credentials out of scripts — sign in interactively. **Pass/fail:** the connection succeeds and `Get-DlpCompliancePolicy` returns without error.

Step 1 — Assess current state (Assess)

1. Run `DLPCODE-Discover.ps1`. Read-only, it reports every **DLP policy and rule**, which **channels** each

covers (Exchange/SharePoint/OneDrive/Teams/Endpoint), each policy's **mode** (test/enforce), and a sample of available **sensitive information types** (and a reminder to confirm the **Source Code** classifier and **Endpoint DLP** onboarding in the portal).

1. Run `DLPCODE-Check-Prereqs.ps1` — module/connection, the DLP/role signals, and existing policies —

PASS/GAP per signal.

1. Record every GAP and remediate before configuring. **Pass/fail:** the inventory and prereq report

complete; existing coverage/overlap captured in `Source-Code DLP – Map`.

Step 2 — Design the DLP policy (Design)

1. With security/IP, choose the **detection** (Source Code classifier + custom IP types/keywords), the

channels (email, SharePoint/OneDrive, Teams, endpoint), the **egress actions** to govern (external email, USB/removable-media copy, cloud/browser upload), the **actions** (restrict/block, notify with a policy tip, alert admins), and the **exceptions** (approved repos/tools/partners).

1. Define `Source-Code DLP – Pilot Ring – Stage1` (developer users/devices) and the notification/alert

recipients. Record everything in `Source-Code DLP – Map`.

1. Confirm which capabilities your licensing supports (Endpoint DLP, Source Code classifier); mark any

unsupported channel/condition **deferred**. **Pass/fail:** `Source-Code DLP – Map` is agreed and signed off; what is protected IP and which egress to block is owned by stakeholders, not IT.

Step 3 — Create the policy in test mode (Configure)

Do this **for the pilot ring first**, and **in test mode** — no enforcement yet.

1. **Create the policy + rule (test mode).** Preview then commit:

```
.\DLPCODE-Set-DlpPolicy.ps1 -Action CreatePolicy -PilotUsers "codedlp-pilot@contoso.com" -UseSourceCode
.\DLPCODE-Set-DlpPolicy.ps1 -Action CreatePolicy -PilotUsers "codedlp-pilot@contoso.com" -UseSourceCode
```

This creates `Source-Code & IP DLP – Policy` across the in-scope channels for the pilot users, with `Source-Code Detection Rule` matching the **Source Code** classifier (and add your custom IP types), in **test-without-notifications** mode by default (change to test-with-tips as designed). Verify with `Get-DlpCompliancePolicy` / `Get-DlpComplianceRule`.

1. **Complete detail in the portal.** Purview portal → **Data Loss Prevention** → **Policies** → the policy →

confirm the rule conditions (classifier + custom IP types), the **endpoint egress activities** (USB/upload), policy-tip text, notifications, alerts, and exceptions match `Source-Code DLP – Map`. **Pass/fail:** the

policy exists in **test mode**, covers the intended channels for the pilot ring, and its rule matches source code / IP — enforcing nothing.

Step 4 — Pilot in test mode (Pilot)

1. Let the policy run in test mode on the pilot ring. Generate representative activity — email a code file

externally; attempt a USB copy on an onboarded pilot device; upload code to a personal cloud.

1. Review what the policy **would** match and block (DLP reports / Activity Explorer) across channels and

where **policy tips** would show. Tune the classifier confidence, custom types, and exceptions (approved repos/tools) to remove false positives. **Pass/fail:** the policy detects source code / IP across the intended channels in test mode with an acceptable false-positive rate; discrepancies are recorded for troubleshooting, not worked around.

Step 5 — Validate the pilot (Pilot → Validate)

1. Run `DLPCODE-Validate.ps1`. Read-only, it confirms `Source-Code & IP DLP – Policy` exists, covers the

intended channels, its rule matches the source-code/IP detection, and the policy is in **test mode** (not prematurely enforcing) — PASS/FAIL per gate.

1. Confirm policy tips and admin alerts behave as designed on a real test egress across channels.
2. Complete the **Testing Guide** on the pilot. **Pass/fail:** all Testing-Guide gates pass on

`Source-Code DLP – Pilot Ring – Stage1`.

Step 6 — Enforce and expand (Expand)

1. **Move to enforce deliberately** — only after test-mode is validated and stakeholders approve:

```
.\DLPCODE-Set-DlpPolicy.ps1 -Action Enforce -WhatIf
.\DLPCODE-Set-DlpPolicy.ps1 -Action Enforce -Commit
```

This switches `Source-Code & IP DLP – Policy` to **Enable** (enforce). Communicate the change to developers first; enforcement blocks real egress.

1. **Expand the scope** to all intended developer users/devices and channels once validated.
2. Run `DLPCODE-Report.ps1 -CsvPath .\CodeDlpReport.csv` to produce `Source-Code DLP – Coverage Report` —

each policy/rule, its channels, mode, and match conditions — as the management evidence artifact, and re-run it on a schedule. **Pass/fail:** the policy is in enforce mode across the intended channels/scope; the report shows coverage and mode as designed.

Rollback

• **Return to test mode** with `DLPCODE-Set-DlpPolicy.ps1 -Action Test -Commit` (or

`Set-DlpCompliancePolicy ... -Mode TestWithoutNotifications`) to stop enforcement immediately while keeping the policy for tuning.

• **Disable / delete the policy** (`Set-DlpCompliancePolicy ... -Mode Disable`, or remove it) to stop it

entirely.

• **Narrow the scope** back to the pilot ring, or add an **exception** to release an approved repo/tool

that's being blocked. Keep `Source-Code DLP – Map` as the record of the intended state.

Order of operations

`Assess policies + classifiers + endpoint onboarding → design detection + channels + actions + exceptions → create policy in test mode on pilot ring → pilot + tune in test mode → validate → enforce deliberately (approved) → expand scope/channels → report coverage`

5. Testing Guide — Source-Code & IP Protection DLP (Microsoft Purview)

Run after each configuration step. Every test is safe (read-only or a `-WhatIf` preview) except where it asks you to generate representative egress on a pilot user/device — and the policy is in **test mode**, so no real egress is blocked. Validate on `Source-Code DLP – Pilot Ring – Stage1` before enforcing or expanding.

Test 1 — Prerequisites and enablement are in place

Do: Run `DLPCODE-Check-Prereqs.ps1`. **Expected:** ExchangeOnlineManagement present and connected; the DLP/compliance role reported; the DLP capability signal reported; existing policies listed. Every gap (e.g. Endpoint DLP / Source Code classifier unavailable → deferred) is **recorded, not guessed**.

Test 2 — Discovery inventory is complete

Do: Run `DLPCODE-Discover.ps1`. **Expected:** Every DLP policy and rule is listed with which **channels** it covers (Exchange/SharePoint/OneDrive/Teams/Endpoint) and its **mode**; available **sensitive information types** are sampled; the reminder to confirm the **Source Code** classifier and **Endpoint DLP** onboarding is shown.

Test 3 — Create dry-run makes no changes

Do: Run `DLPCODE-Set-DlpPolicy.ps1 -Action CreatePolicy -PilotUsers "codedlp-pilot@contoso.com" -UseSourceCodeClassifier -WhatIf`. **Expected:** It states the policy/rule/channels it *would* create (test mode) plus the reverse action and changes nothing; a follow-up `Get-DlpCompliancePolicy` shows no new policy until `-Commit`.

Test 4 — Policy exists in test mode across the intended channels

Do: After `-Commit`, run `Get-DlpCompliancePolicy -Identity "Source-Code & IP DLP - Policy"` and `Get-DlpComplianceRule`. **Expected:** The policy exists, covers the intended channels

(email, SharePoint/OneDrive, Teams, and — where licensed — Endpoint), its rule matches the Source Code classifier (+ custom IP types), and the mode is **test** — enforcing nothing.

Test 5 — The rule detects source code

Do: Create test content that the Source Code classifier should match (a representative code file) and route it through an in-scope channel (e.g. attach to an internal email or upload to a pilot site). **Expected:** The DLP reports / Activity Explorer show a **match** in test mode — the policy detects source code. (Trainable-classifier matching is probabilistic; verify behavior against current Microsoft guidance.)

Test 6 — Endpoint egress is evaluated (where licensed)

Do: On an **onboarded** pilot device, attempt to copy a matching code file to **USB/removable media** and to upload it via an unmanaged browser. **Expected:** In test mode the policy records what it **would** restrict for the USB-copy and upload activities (and where a **policy tip** would appear) — without blocking. If Endpoint DLP is out of plan, this test is recorded **deferred**.

Test 7 — External email egress is evaluated

Do: Email a matching code file to an **external** recipient from a pilot user. **Expected:** In test mode the policy records what it **would** restrict for the external-email condition — without blocking the send.

Test 8 — Validation gate passes on the pilot (test mode)

Do: Run `DLPCODE-Validate.ps1`. **Expected:** PASS for each gate — the policy exists, covers the intended channels, the rule matches the source-code/IP detection, and the policy is in **test mode** (not prematurely enforcing). Any FAIL names the specific issue.

Test 9 — False positives are tuned out

Do: Review test-mode matches for legitimate developer activity (build tools, approved-repo pushes, partner sharing) that shouldn't match; add an **exception** or adjust the classifier confidence/conditions; re-review. **Expected:** After tuning, legitimate developer workflow no longer matches while real source-code/IP egress still does — an acceptable false-positive rate before enforcement.

Test 10 — Enforcement blocks as designed (deliberate, after approval)

Do: Only after validation and approval: enforce with `-Commit`, then attempt a matching egress (external email / USB copy / upload) as a normal pilot user/device. **Expected:** The egress is **restricted/blocked** per the rule, the user sees the **policy tip**, and an **admin alert** fires. An approved/excepted path still works. (Verify against your configuration.)

Test 11 — Reporting evidence is produced

Do: Run `DLPCODE-Report.ps1 -CsvPath .\CodeDlpReport.csv`. **Expected:** `Source-Code DLP – Coverage Report` lists each policy/rule, its channels, mode, and match conditions — usable as the management evidence artifact and re-runnable on a schedule.

Pass/fail summary

All gates green on `Source-Code DLP – Pilot Ring – Stage1` — policy present in test mode, detecting source code / IP across the intended channels with false positives tuned out, policy tips and alerts behaving as designed — before moving to enforce or expanding. Enforcement is enabled only as a deliberate, approved step; any deferred (Endpoint-DLP/classifier/licensing-gated) capability is recorded explicitly, not silently skipped.

6. Troubleshooting — Source-Code & IP Protection DLP (Microsoft Purview)

Symptoms, likely causes, and fixes for source-code/IP DLP. Stay in **test mode** while diagnosing — reproduce on `Source-Code DLP – Pilot Ring – Stage1` before enforcing or expanding, and never flip a policy to enforce to "test" a theory against real developer workflow.

The policy matches nothing / no matches in reports

• **Propagation delay.** New DLP policies take time (often up to ~1 hour, longer at scale) to activate;

endpoint policies also need the device to sync. Confirm the policy exists and wait before concluding failure.

• **Source Code classifier not on the rule.** The Source Code trainable classifier is added by **classifier

reference in the portal** (the script scaffolds the policy/rule and custom SITs; the classifier is completed in the portal). Confirm the rule actually references the classifier.

• **Custom IP types too specific / wrong name.** Verify custom SIT names (`DLPCODE-Discover.ps1` lists info

types) and that your test content matches them.

• **Channel not in scope.** Confirm the egress channel you're testing (email/SharePoint/OneDrive/Teams/

Endpoint) is in the policy's locations.

Endpoint (USB/upload) activity isn't detected

• **Device not onboarded.** Endpoint DLP requires the device to be **onboarded** (shared with Defender for

Endpoint). Confirm the pilot device shows in Purview → Settings → Endpoint DLP.

• **Egress activity not enabled.** The specific activities (copy to removable media, upload to cloud/

unmanaged browser) must be enabled in Endpoint DLP settings and referenced by the rule. Confirm both.

• **Licensing.** Endpoint DLP requires **E5 / E5 Compliance**. `DLPCODE-Check-Prereqs.ps1` flags the DLP

signal; where endpoint is out of plan, record it **deferred** and rely on email/collaboration channels.

• **Sync lag.** Endpoint policy changes take time to reach the device. Have the device sync and retry.

The policy blocks legitimate developer activity (false positives)

• **Classifier confidence too low / condition too broad.** Raise the Source Code classifier confidence or

tighten the conditions so comments, config, and docs don't trigger it.

• **Add an exception for approved egress.** Except approved repositories, build systems, CI/CD, and

sanctioned partner-sharing paths. This is expected tuning — do it in **test mode** before enforcing.

• **Still in test mode?** *Would-block* entries in test mode are impact reporting, not live blocks — which

is exactly why you pilot in test mode first.

Enforcement is blocking developers unexpectedly

• **Enforced too early.** Enforcement should follow a validated test-mode pilot. If it's blocking

legitimate work, return to test mode immediately: `DLPCODE-Set-DlpPolicy.ps1 -Action Test -Commit`, then tune.

• **Missing exception.** An approved repo/tool wasn't excepted. Add the exception, re-validate in test

mode, then re-enforce.

Policy tips don't appear

• **Notifications off / wrong mode.** Tips show in **enforce** mode and **test-with-notifications**, not in

test-without-notifications. Confirm the mode and that the rule's policy tip / user notification is enabled. Endpoint tips also depend on device support.

• **Tip text not configured.** Complete the tip text in the portal (not asserted by the script).

`DLPCODE-Set-DlpPolicy.ps1 -Commit` fails

• **Not connected / wrong session.** Ensure `Connect-IPPSSession` succeeded

(`Get-Command Get-DlpCompliancePolicy` resolves).

• **Insufficient role.** Creating DLP policies needs Compliance Administrator / DLP Compliance Management.

Confirm in Purview → Roles & scopes / Permissions.

• **Parameter/location shape by module version.** `-EndpointDlpLocation`, `-TeamsLocation`,

`-ContentContainsSensitiveInformation`, and `-Mode` values vary across ExchangeOnlineManagement versions. Verify against your version; update the module if it's far behind.

• **Classifier not addable via cmdlet.** The Source Code trainable classifier is typically added by

reference in the portal rather than a simple parameter — complete it there and re-validate.

`DLPCODE-Validate.ps1` reports a channel/mode wrong

• **Module-version property shape.** Location and mode properties are exposed under different names across

module versions. Confirm the intended state in the portal (Data Loss Prevention → the policy) and treat the validator line as informational where the portal disagrees; note the module version.

When to stop and escalate

• A required capability is unavailable on your licensing/tenant (Endpoint DLP, the Source Code classifier)

— record it **deferred** and route the licensing decision to the owner; do not design around an unavailable feature.

• **What is protected IP and which egress is acceptable** is a stakeholder determination (security/IP/

legal), not a technical fix — capture it in `Source-Code DLP – Map` and defer the determination.

• If match or enforcement behavior can't be reconciled with the policy, keep it in test mode on the pilot

ring, gather `DLPCODE-Validate.ps1` + `DLPCODE-Report.ps1` output as evidence, and raise with Microsoft support before enforcing or expanding.

7. Best Practices — Source-Code & IP Protection DLP (Microsoft Purview)

• **Test mode first; enforce deliberately.** The single most important discipline: create the policy in

test mode, review what it *would* match and block across channels on real developer activity, tune it, and enforce only after approval. Enforcing an untuned code/IP policy blocks legitimate development and erodes trust fast.

• **Detect on content, not extension.** Renaming a file or pasting a snippet defeats extension rules. Lead

with the **Source Code trainable classifier** and custom IP types so detection follows what the content *is*, wherever it goes.

• **What is protected IP is a security/legal determination.** The classifier plus custom IP types come from

security/IP/legal — which code-names, markings, and identifiers matter. Configure the conditions they specify; capture the determinations and approvers in `Source-Code DLP – Map`.

• **Close the endpoint gap where you can.** Email and SharePoint DLP don't see a USB copy or a browser

upload — Endpoint DLP does. Where E5/onboarding allows, include the endpoint channel; where it doesn't, scope to email/collaboration and record endpoint coverage as **deferred** rather than pretending it's covered.

• **Except approved developer egress explicitly.** Build systems, approved repositories, CI/CD, and

sanctioned partner sharing are legitimate egress. Except them deliberately so protection lands on unsanctioned exfiltration, not on the daily workflow.

• **Tune the classifier confidence to your false-positive tolerance.** Trainable classifiers are

probabilistic. Use the test-mode pilot to find the confidence and conditions that catch real code/IP without flagging comments, config, or documentation.

• **Pilot on a developer ring first.** Prove the policy on `Source-Code DLP – Pilot Ring – Stage1` — real

developers/devices — before scoping to all engineering. Match behavior across channels is easier to reason about on a bounded ring.

• **Use policy tips to teach.** A tip at the moment of egress ("this looks like source code — external

sharing is restricted") prevents mistakes and trains developers. Design the tip text deliberately; it's the most visible part of the policy.

• **Coordinate with, don't replace, developer-platform security.** DLP governs egress through M365 and

endpoints; it doesn't secure the Git server or scan repos for secrets. Deploy them together and keep each doing its own job.

• **Keep automation safe-by-default and secret-free.** Run discovery, validation, and reporting read-only;

gate every write behind `-WhatIf` → `-Commit`; enforcement behind its own explicit `-Action Enforce`. Sign in interactively; no credentials or secrets in a script or a document.

• **Re-report and re-tune on a schedule.** New projects, code-names, and egress patterns appear constantly.

Re-run `DLPCODE-Report.ps1` and review matches periodically so `Source-Code DLP – Coverage Report` stays a live evidence artifact and the policy stays accurate.

• **Keep determinations with their owners.** The solution configures and evidences code/IP DLP; what is

protected IP, which egress is acceptable, and any legal obligation are determinations for security, IP, and legal. Keep that separation explicit.

8. Deliverables Summary — Source-Code & IP Protection DLP (Microsoft Purview)

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box
1. Solution Overview	DLP + classifiers detecting and protecting source code / IP across email, collaboration, and endpoints; test mode first, enforce deliberately; how DLP differs from repo security (the why)
2. Requirements Guide	Every prerequisite with Verify / Implement-if-missing

Document	Purpose
Global Requirements (Purview)	Category-wide prerequisites, published once
3. Implementation Guide	Exact, ordered steps: assess policies + classifiers + endpoint onboarding, design detection + channels + actions, create the policy in test mode on the pilot ring, pilot + tune, validate, enforce deliberately, expand
5. Testing Guide	Validation procedures with pass/fail gates
6. Troubleshooting Guide	Field issues and remediation
7. Best Practices	Field-proven recommendations

Named artifacts

Artifact	Name
DLP policy	`Source-Code & IP DLP – Policy`
Detection rule	`Source-Code Detection Rule`
Pilot ring	`Source-Code DLP – Pilot Ring – Stage1`
Scope/design map	`Source-Code DLP – Map`
Coverage report	`Source-Code DLP – Coverage Report`

PowerShell scripts

Script	What it does
`DLPCODE-Discover.ps1`	Reports current state: every DLP policy and rule, which channels each covers (Exchange/SharePoint/OneDrive/Teams/Endpoint) and its mode, and a sample of available sensitive information types (with a Source Code classifier / Endpoint DLP reminder) — Security & Compliance PowerShell; read-only
`DLPCODE-Check-Prereqs.ps1`	Prerequisite/health helper: module, connection, the DLP/compliance role signal, and existing policies — PASS/GAP per signal; read-only

Script	What it does
`DLPCODE-Set-DlpPolicy.ps1`	Configuration/Remediation: creates `Source-Code & IP DLP – Policy` + `Source-Code Detection Rule` across the in-scope channels for the pilot ring **in test mode** (`-Action CreatePolicy`, `-UseSourceCodeClassifier`), and — as deliberate separate steps — switches to **enforce** (`-Action Enforce`) or back to **test** (`-Action Test`); **defaults to** `-WhatIf` (prints the plan + reverse action), applies only with `-Commit`; no secrets in code
`DLPCODE-Validate.ps1`	Validation: PASS/FAIL that the policy exists, covers the intended channels, its rule matches the source-code/IP detection, and the policy is in **test mode** (not prematurely enforcing) — read-only
`DLPCODE-Report.ps1`	Coverage report — each policy/rule, its channels, mode, and match conditions — written to CSV as `Source-Code DLP – Coverage Report`; read-only

What the buyer receives

- This implementation guide as a Word document and PDF.
- All five PowerShell scripts in a single ZIP.
- A repeatable method following the model: Assess → Design → Configure → Pilot → Validate → Expand.

Outcome

Source code and IP governed from Purview — detected by classifier and custom types and restricted from leaving via email, cloud upload, and USB across the in-scope channels — proven in test mode on a developer pilot ring before any enforcement, with the determination of what is protected IP and which egress is acceptable left to the security, IP, and legal owners who make it.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

DLPCODE-Check-Prereqs.ps1

Purpose: Confirms the environment is ready to configure code/IP DLP: the ExchangeOnlineManagement module and S&C PowerShell connection, whether the DLP cmdlets and sensitive info types are available, and existing DLP policies. Prints the manual checks (Source Code classifier, Endpoint DLP onboarding). Read-only. HOW TO RUN .\DLPCODE-Check-Prereqs.ps1 PowerShell 5.1+/7+. Connect first with Connect-IPPSSession. No secret is stored in the script. EXPECTED RESULTS A PASS/GAP line per signal plus a manual checklist. GAPs are recorded against the Requirements checklist and remediated before configuring. Makes no change. .NOTES Purview — Source-Code & IP Protection DLP. Read-only. Verify your licensing/tenant: core DLP is E3; Endpoint DLP + Source Code classifier require E5/E5 Compliance. > [CmdletBinding()] param()

How to run: .\DLPCODE-Check-Prereqs.ps1 PowerShell 5.1+/7+. Connect first with Connect-IPPSSession. No secret is stored in the script. EXPECTED RESULTS A PASS/GAP line per signal plus a manual checklist. GAPs are recorded against the Requirements checklist and remediated before configuring. Makes no change. .NOTES Purview — Source-Code & IP Protection DLP. Read-only. Verify your licensing/tenant: core DLP is E3; Endpoint DLP + Source Code classifier require E5/E5 Compliance. > [CmdletBinding()] param()

DLPCODE-Discover.ps1

Purpose: Reports the current DLP state for code/IP protection: every DLP policy and rule, which channels each covers (Exchange / SharePoint / OneDrive / Teams / Endpoint), each policy's mode (test/enforce), and a sample of available sensitive information types. Reminds you to confirm the Source Code classifier and Endpoint DLP onboarding in the portal (not readable here). Read-only — makes no change. HOW TO RUN .\DLPCODE-Discover.ps1 PowerShell 5.1+/7+. Connect first with Connect-IPPSSession. No secret is stored in the script. EXPECTED RESULTS A policy/rule inventory with channels and mode, plus available sensitive info types — captured into 'Source-Code DLP - Map'. Makes no configuration change. .NOTES Purview — Source-Code & IP Protection DLP. Read-only discovery. Verify your licensing/tenant: Endpoint DLP + Source Code classifier require E5; property names vary by module version. > [CmdletBinding()] param()

How to run: .\DLPCODE-Discover.ps1 PowerShell 5.1+/7+. Connect first with Connect-IPPSSession. No secret is stored in the script. EXPECTED RESULTS A policy/rule inventory with channels and mode, plus available sensitive info types — captured into 'Source-Code DLP - Map'. Makes no configuration change. .NOTES Purview — Source-Code & IP Protection DLP. Read-only discovery. Verify your licensing/tenant: Endpoint DLP + Source Code classifier require E5; property names vary by module version. > [CmdletBinding()] param()

DLPCODE-Report.ps1

Purpose: Builds 'Source-Code DLP - Coverage Report' — for each DLP policy/rule: which channels it covers (Exchange/SharePoint/OneDrive/Teams/Endpoint), its mode (test/enforce), and its rules — as the management evidence artifact. Read-only; writing a CSV is opt-in. HOW TO RUN .\DLPCODE-Report.ps1 .\DLPCODE-Report.ps1 -CsvPath .\CodeDlpReport.csv PowerShell 5.1+/7+. Connect first with

Connect-IPPSSession. No secret is stored in the script. EXPECTED RESULTS A table (and optional CSV) of each policy/rule with channels and mode — re-runnable on a schedule. Makes no configuration change. .NOTES Purview — Source-Code & IP Protection DLP. Read-only reporting. Verify your licensing/tenant; property names vary by module version. > [CmdletBinding()] param([string]$CsvPath)

How to run: .\DLPCODE-Report.ps1 .\DLPCODE-Report.ps1 -CsvPath .\CodeDlpReport.csv PowerShell 5.1+/7+. Connect first with Connect-IPPSSession. No secret is stored in the script. EXPECTED RESULTS A table (and optional CSV) of each policy/rule with channels and mode — re-runnable on a schedule. Makes no configuration change. .NOTES Purview — Source-Code & IP Protection DLP. Read-only reporting. Verify your licensing/tenant; property names vary by module version. > [CmdletBinding()] param([string]$CsvPath)

DLPCODE-Set-DlpPolicy.ps1

Purpose: Automates the DLP-policy lifecycle SAFELY. -Action CreatePolicy creates 'Source-Code & IP DLP - Policy' + 'Source-Code Detection Rule' across the in-scope channels (Exchange/SharePoint/OneDrive/ Teams/Endpoint) for the pilot users, in TEST mode (enforces nothing). -Action Enforce switches the policy to Enable (enforce) — a DELIBERATE step run only after test-mode is validated and approved. -Action Test returns it to test mode. DEFAULTS TO -WhatIf: prints exactly what it would do plus the reverse action and changes nothing; applies only with -Commit. No secrets in code. HOW TO RUN .\DLPCODE-Set-DlpPolicy.ps1 -Action CreatePolicy -PilotUsers "codedlp-pilot@contoso.com" -UseSourceCodeClassifier -WhatIf .\DLPCODE-Set-DlpPolicy.ps1 -Action CreatePolicy -PilotUsers "codedlp-pilot@contoso.com" -UseSourceCodeClassifier -Commit .\DLPCODE-Set-DlpPolicy.ps1 -Action Enforce -Commit .\DLPCODE-Set-DlpPolicy.ps1 -Action Test -Commit PowerShell 5.1+/7+. Connect first with Connect-IPPSSession (modern-auth/MFA admin). EXPECTED RESULTS -WhatIf: the plan + reverse command, no change. CreatePolicy -Commit: policy + rule exist in test mode across the chosen channels (verify with Get-DlpCompliancePolicy). Enforce/Test -Commit: policy Mode changes accordingly. Complete the Source Code classifier + endpoint egress detail in the portal. .NOTES Purview — Source-Code & IP Protection DLP. Safe-by-default; enforcement behind its own action. Verify your licensing/tenant: Endpoint DLP + Source Code classifier require E5; confirm the classifier's exact reference in the portal. > [CmdletBinding()] param([ValidateSet('CreatePolicy','Enforce','Test')] [string]$Action = 'CreatePolicy', [string]$PolicyName = 'Source-Code & IP DLP - Policy', [string]$RuleName = 'Source-Code Detection Rule', [string[]]$PilotUsers = @(), # UPNs / groups for the pilot ring (Exchange/Teams/Endpoint scoping) [switch]$IncludeEndpoint, # include Endpoint DLP (requires E5 + onboarding) [switch]$UseSourceCodeClassifier, # reference the built-in Source Code trainable classifier [string[]]$CustomSensitiveTypes = @(), # exact custom IP SIT names (see DLPCODE-Discover) [switch]$WhatIf, [switch]$Commit)

How to run: .\DLPCODE-Set-DlpPolicy.ps1 -Action CreatePolicy -PilotUsers "codedlp-pilot@contoso.com" -UseSourceCodeClassifier -WhatIf .\DLPCODE-Set-DlpPolicy.ps1 -Action CreatePolicy -PilotUsers "codedlp-pilot@contoso.com" -UseSourceCodeClassifier -Commit .\DLPCODE-Set-DlpPolicy.ps1 -Action Enforce -Commit .\DLPCODE-Set-DlpPolicy.ps1 -Action Test -Commit PowerShell 5.1+/7+. Connect first with Connect-IPPSSession (modern-auth/MFA admin). EXPECTED RESULTS -WhatIf: the plan + reverse command, no change. CreatePolicy -Commit: policy + rule exist in test mode across the chosen channels (verify with Get-DlpCompliancePolicy). Enforce/Test -Commit: policy Mode changes accordingly. Complete the Source Code classifier + endpoint egress detail in the portal. .NOTES Purview — Source-Code & IP Protection DLP. Safe-by-default; enforcement behind its own action. Verify your licensing/tenant: Endpoint DLP + Source Code classifier require E5; confirm the classifier's exact reference in the portal. > [CmdletBinding()] param(

[ValidateSet('CreatePolicy','Enforce','Test')] [string]$Action = 'CreatePolicy', [string]$PolicyName = 'Source-Code & IP DLP - Policy', [string]$RuleName = 'Source-Code Detection Rule', [string[]]$PilotUsers = @(), # UPNs / groups for the pilot ring (Exchange/Teams/Endpoint scoping) [switch]$IncludeEndpoint, # include Endpoint DLP (requires E5 + onboarding) [switch]$UseSourceCodeClassifier, # reference the built-in Source Code trainable classifier [string[]]$CustomSensitiveTypes = @(), # exact custom IP SIT names (see DLPCODE-Discover) [switch]$WhatIf, [switch]$Commit)

DLPCODE-Validate.ps1

Purpose: The pilot/expansion gate. Confirms PASS/FAIL that 'Source-Code & IP DLP - Policy' exists, covers the intended channels (Exchange/SharePoint/OneDrive/Teams/Endpoint), has its detection rule, and — for the pilot — is in TEST mode (not prematurely enforcing). Read-only. Run on the pilot before enforcing, and after each expansion wave. HOW TO RUN .\DLPCODE-Validate.ps1 -ExpectEnforce:$false -ExpectEndpoint:$true PowerShell 5.1+/7+. Connect first with Connect-IPPSSession. No secret is stored in the script. EXPECTED RESULTS A PASS/FAIL line per gate; overall PASS only when the policy matches the intended design. During the pilot, the policy being in Enable (enforce) mode FAILS the "still in test mode" gate. .NOTES Purview — Source-Code & IP Protection DLP. Read-only gate. Verify your licensing/tenant; property/mode names vary by module version. Endpoint coverage requires E5 + onboarding. > [CmdletBinding()] param([string]$PolicyName = 'Source-Code & IP DLP - Policy', [string]$RuleName = 'Source-Code Detection Rule', [bool]$ExpectEnforce = $false, # pilot expects test mode ($false); after approval set $true [bool]$ExpectEndpoint = $false # set $true if Endpoint DLP is in scope (E5 + onboarding))

How to run: .\DLPCODE-Validate.ps1 -ExpectEnforce:$false -ExpectEndpoint:$true PowerShell 5.1+/7+. Connect first with Connect-IPPSSession. No secret is stored in the script. EXPECTED RESULTS A PASS/FAIL line per gate; overall PASS only when the policy matches the intended design. During the pilot, the policy being in Enable (enforce) mode FAILS the "still in test mode" gate. .NOTES Purview — Source-Code & IP Protection DLP. Read-only gate. Verify your licensing/tenant; property/mode names vary by module version. Endpoint coverage requires E5 + onboarding. > [CmdletBinding()] param([string]$PolicyName = 'Source-Code & IP DLP - Policy', [string]$RuleName = 'Source-Code Detection Rule', [bool]$ExpectEnforce = $false, # pilot expects test mode ($false); after approval set $true [bool]$ExpectEndpoint = $false # set $true if Endpoint DLP is in scope (E5 + onboarding))